

Introducción al Tidyverse

Sebastián

2026-05-13

Tabla de contenidos

1. Tidyverse	2
2. Paquetes principales del tidyverse	3
2.1. Nueve paquetes, infinitas posibilidades...	3
3. Instalación y carga de paquetes	3
4. El Tibble	4
4.1. Una reimaginación del dataframe	4
4.2. Todos los tibbles son dataframes	4
4.2.1. Pero no todos los dataframes son tibbles	5
4.3. Convertir un dataframe a un tibble	5
4.3.1. ¡Cuidado al convertir dataframes en tibbles!	5
4.3.2. Convertir un dataframe a un tibble, de forma segura :)	6
4.4. Tribble para crear tibbles	7
4.5. Acceder a datos en un tibble	7
5. Readr	8
5.1. Escritura de archivos	8
5.2. Lectura de archivos	9
5.3. Algunas de las opciones más útiles de readr	10
5.4. Lectura de otros archivos comunes	11
5.4.1. Excel	11
5.4.2. Google sheets	12
6. Tidyr	12
6.1. Filosofía del tidyverse	12
6.2. Ejemplo de datos desordenados	12
6.3. Transformación a lo largo: pivot_longer()	13
6.3.1. Ahora es más fácil de graficar	14
6.4. Transformación a lo ancho: pivot_wider()	15
6.5. Otro ejemplo de datos desordenados	16

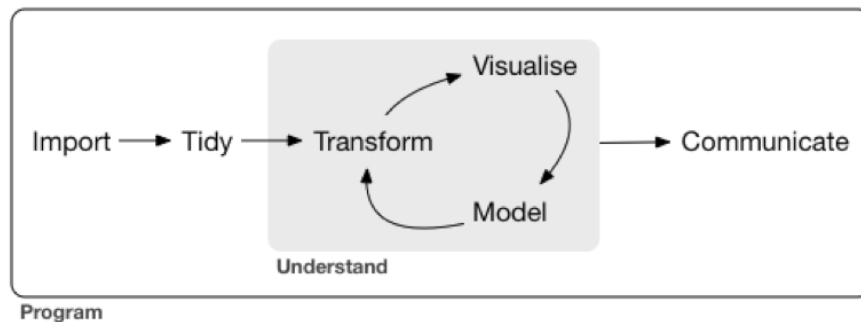
6.6. Separación a lo largo: <code>separate_longer_delim()</code>	16
6.7. Separación a lo ancho: <code>separate_wider_delim()</code>	17
6.8. Unir variables	17
7. Purrr	18
7.1. ¿Por qué programación funcional?	18
7.2. Con programación funcional podemos sustituir ciclos	18
7.2.1. También podemos crear una función con la misma utilidad	18
7.3. <code>map()</code>	18
7.4. <code>walk()</code>	19
7.5. <code>map2</code> y <code>walk2</code>	20
7.6. <code>pmap</code> y <code>pwalk</code>	21
7.7. Seguimiento de progreso	22
8. Bibliografía:	22

1. Tidyverse



El tidyverse es una colección de paquetes de R diseñados para la ciencia de datos, con una filosofía bien definida. Todos los paquetes comparten una filosofía de diseño subyacente, una gramática y estructuras de datos comunes.

Tidyverse



2. Paquetes principales del tidyverse

2.1. Nueve paquetes, infinitas posibilidades...

ggplot2: Creación de gráficos basada en “The Grammar of Graphics”.

dplyr: Herramienta para la manipulación y transformación de datos.

tidyr: Permite organizar y limpiar datos (tidy data).

readr: Facilita la importación y exportación de datos.

purrr: Simplifica tareas repetitivas con programación funcional.

tibble: Versión moderna de los data.frames.

stringr: Funciones para trabajar de forma sencilla con texto.

forcats: Herramientas para manipular variables categóricas (factores).

lubridate: Facilita el manejo y la manipulación de fechas y horas.

3. Instalación y carga de paquetes

```
# install.packages("tidyverse")  
library(tidyverse)
```

i Nota

El **tidyverse** también incluye otros paquetes más especializados. Estos no se cargan automáticamente con `library(tidyverse)`, por lo que deben importarse individualmente usando `library()`.

4. El Tibble

4.1. Una reimaginación del dataframe

```
data <- data.frame(a = 1:3, b = letters[1:3], c = Sys.Date() - 1:3)
data
```

```
  a b      c
1 1 a 2026-05-12
2 2 b 2026-05-11
3 3 c 2026-05-10
```

```
as_tibble(data)
```

```
# A tibble: 3 x 3
  a b      c
<int> <chr> <date>
1     1 a 2026-05-12
2     2 b 2026-05-11
3     3 c 2026-05-10
```

Más información y mejor presentación

- Es un tibble
- Dimensiones del tibble: 3x3
- Naturaleza de las variables: <int>, <chr>, <date>

4.2. Todos los tibbles son dataframes

```
is.data.frame(data)
```

```
[1] TRUE
```

```
is.data.frame(as_tibble(data))
```

```
[1] TRUE
```

4.2.1. Pero no todos los dataframes son tibbles

```
is_tibble(data)
```

```
[1] FALSE
```

```
is_tibble(as_tibble(data))
```

```
[1] TRUE
```

4.3. Convertir un dataframe a un tibble

```
as_tibble(data)
```

```
# A tibble: 3 x 3
      a b      c
  <int> <chr> <date>
1     1 a 2026-05-12
2     2 b 2026-05-11
3     3 c 2026-05-10
```

```
data %>% as_tibble()
```

```
# A tibble: 3 x 3
      a b      c
  <int> <chr> <date>
1     1 a 2026-05-12
2     2 b 2026-05-11
3     3 c 2026-05-10
```

4.3.1. ¡Cuidado al convertir dataframes en tibbles!

```
head(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

```
as_tibble(mtcars)
```

```
# A tibble: 32 x 11
      mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1    21     6   160    110   3.9    2.62  16.5     0     1     4     4
```

```

2  21      6  160    110  3.9   2.88  17.0    0    1    4    4
3  22.8    4  108     93  3.85  2.32  18.6    1    1    4    1
4  21.4    6  258    110  3.08  3.22  19.4    1    0    3    1
5  18.7    8  360    175  3.15  3.44  17.0    0    0    3    2
6  18.1    6  225    105  2.76  3.46  20.2    1    0    3    1
7  14.3    8  360    245  3.21  3.57  15.8    0    0    3    4
8  24.4    4  147.    62  3.69  3.19  20      1    0    4    2
9  22.8    4  141.    95  3.92  3.15  22.9    1    0    4    2
10 19.2    6  168.   123  3.92  3.44  18.3    1    0    4    4
# i 22 more rows

```

⚠ ¡Cuidado!

Los tibbles **no tienen nombres de filas**. Convertir un dataframe en un tibble eliminará los nombres de filas.

4.3.2. Convertir un dataframe a un tibble, de forma segura :)

```
as_tibble(x = mtcars, rownames = "model") %>% print(n = 5)
```

```

# A tibble: 32 x 12
  model      mpg  cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21    6   160   110   3.9   2.62  16.5    0    1    4     4
2 Mazda RX4 W~   21    6   160   110   3.9   2.88  17.0    0    1    4     4
3 Datsun 710     22.8    4   108    93   3.85  2.32  18.6    1    1    4     1
4 Hornet 4 Dr~   21.4    6   258   110   3.08  3.22  19.4    1    0    3     1
5 Hornet Spor~   18.7    8   360   175   3.15  3.44  17.0    0    0    3     2
# i 27 more rows

```

```
mtcars %>% as_tibble(rownames = "model") %>% print(n = 5)
```

```

# A tibble: 32 x 12
  model      mpg  cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21    6   160   110   3.9   2.62  16.5    0    1    4     4
2 Mazda RX4 W~   21    6   160   110   3.9   2.88  17.0    0    1    4     4
3 Datsun 710     22.8    4   108    93   3.85  2.32  18.6    1    1    4     1
4 Hornet 4 Dr~   21.4    6   258   110   3.08  3.22  19.4    1    0    3     1
5 Hornet Spor~   18.7    8   360   175   3.15  3.44  17.0    0    0    3     2
# i 27 more rows

```

```
mtcars %>% rownames_to_column(var = "model") %>% as_tibble() %>% print(n = 5)
```

```

# A tibble: 32 x 12
  model      mpg  cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>

```

```

1 Mazda RX4      21      6  160  110  3.9  2.62  16.5    0    1    4    4
2 Mazda RX4 W~   21      6  160  110  3.9  2.88  17.0    0    1    4    4
3 Datsun 710     22.8    4  108   93  3.85  2.32  18.6    1    1    4    1
4 Hornet 4 Dr~   21.4    6  258  110  3.08  3.22  19.4    1    0    3    1
5 Hornet Spor~   18.7    8  360  175  3.15  3.44  17.0    0    0    3    2
# i 27 more rows

```

Misma solución, diferentes maneras de abordar el problema.

4.4. Tribble para crear tibbles

```

tibble(
  x = c("a", "b"),
  y = c(2, 1),
  z = c(3.6, 8.5)
)

# A tibble: 2 x 3
  x         y         z
  <chr> <dbl> <dbl>
1 a         2     3.6
2 b         1     8.5

```

Creación de tibbles usando una disposición fila por fila más fácil de leer.

```

tribble(
  ~x, ~y, ~z,
  "a", 2, 3.6,
  "b", 1, 8.5
)

# A tibble: 2 x 3
  x         y         z
  <chr> <dbl> <dbl>
1 a         2     3.6
2 b         1     8.5

```

4.5. Acceder a datos en un tibble

Tan fácil como hacerlo en un dataframe

```

tibble_mtcars <- as_tibble(mtcars, rownames = "model")
tibble_mtcars$model

[1] "Mazda RX4"           "Mazda RX4 Wag"       "Datsun 710"
[4] "Hornet 4 Drive"      "Hornet Sportabout"   "Valiant"
[7] "Duster 360"          "Merc 240D"           "Merc 230"
[10] "Merc 280"            "Merc 280C"           "Merc 450SE"

```

```
[13] "Merc 450SL"      "Merc 450SLC"      "Cadillac Fleetwood"
[16] "Lincoln Continental" "Chrysler Imperial" "Fiat 128"
[19] "Honda Civic"      "Toyota Corolla"   "Toyota Corona"
[22] "Dodge Challenger" "AMC Javelin"      "Camaro Z28"
[25] "Pontiac Firebird"  "Fiat X1-9"        "Porsche 914-2"
[28] "Lotus Europa"     "Ford Pantera L"   "Ferrari Dino"
[31] "Maserati Bora"    "Volvo 142E"
```

```
tibble_mtcars[1,]
```

```
# A tibble: 1 x 12
```

```
  model      mpg  cyl  disp   hp  drat    wt  qsec    vs    am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4    21     6   160   110   3.9   2.62  16.5     0     1     4     4
```

```
tibble_mtcars[,1]
```

```
# A tibble: 32 x 1
```

```
  model
  <chr>
1 Mazda RX4
2 Mazda RX4 Wag
3 Datsun 710
4 Hornet 4 Drive
5 Hornet Sportabout
6 Valiant
7 Duster 360
8 Merc 240D
9 Merc 230
10 Merc 280
# i 22 more rows
```

```
tibble_mtcars[1,1]
```

```
# A tibble: 1 x 1
```

```
  model
  <chr>
1 Mazda RX4
```

5. Readr

5.1. Escritura de archivos

```
head(mtcars)
```

```
      mpg  cyl  disp  hp  drat    wt  qsec vs  am  gear  carb
Mazda RX4    21.0    6  160  110  3.90  2.620  16.46  0   1    4    4
```


Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

⚠ ¡Cuidado!

Estas funciones **no incluyen nombres de filas** en los archivos creados. Por lo que es importante considerarlo antes de guardar los archivos.

```
tibble_mtcars <- as_tibble(x = mtcars, rownames = "model")
write_tsv(tibble_mtcars, "tibble_mtcars.tsv")
write_csv(tibble_mtcars, "tibble_mtcars.csv")
```

5.2. Lectura de archivos

```
read_tsv("tibble_mtcars.tsv")
```

```
# A tibble: 32 x 12
  model      mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160    110   3.9   2.62  16.5     0     1     4     4
2 Mazda RX4 ~    21     6  160    110   3.9   2.88  17.0     0     1     4     4
3 Datsun 710    22.8     4  108     93   3.85   2.32  18.6     1     1     4     1
4 Hornet 4 D~   21.4     6  258    110   3.08   3.22  19.4     1     0     3     1
5 Hornet Spo~  18.7     8  360    175   3.15   3.44  17.0     0     0     3     2
6 Valiant      18.1     6  225    105   2.76   3.46  20.2     1     0     3     1
7 Duster 360   14.3     8  360    245   3.21   3.57  15.8     0     0     3     4
8 Merc 240D    24.4     4  147.     62   3.69   3.19   20      1     0     4     2
9 Merc 230     22.8     4  141.     95   3.92   3.15  22.9     1     0     4     2
10 Merc 280    19.2     6  168.    123   3.92   3.44  18.3     1     0     4     4
# i 22 more rows
```

```
read_csv("tibble_mtcars.csv")
```

```
# A tibble: 32 x 12
  model      mpg   cyl  disp    hp  drat    wt  qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160    110   3.9   2.62  16.5     0     1     4     4
2 Mazda RX4 ~    21     6  160    110   3.9   2.88  17.0     0     1     4     4
3 Datsun 710    22.8     4  108     93   3.85   2.32  18.6     1     1     4     1
4 Hornet 4 D~   21.4     6  258    110   3.08   3.22  19.4     1     0     3     1
5 Hornet Spo~  18.7     8  360    175   3.15   3.44  17.0     0     0     3     2
6 Valiant      18.1     6  225    105   2.76   3.46  20.2     1     0     3     1
7 Duster 360   14.3     8  360    245   3.21   3.57  15.8     0     0     3     4
```

```

8 Merc 240D      24.4      4 147.      62 3.69 3.19 20      1      0      4      2
9 Merc 230       22.8      4 141.      95 3.92 3.15 22.9    1      0      4      2
10 Merc 280      19.2      6 168.     123 3.92 3.44 18.3    1      0      4      4
# i 22 more rows

```

5.3. Algunas de las opciones más útiles de readr

Lectura de archivos más discreta con la opción `show_col_types = FALSE`

```
read_tsv("tibble_mtcars.tsv", show_col_types = FALSE)
```

```

# A tibble: 32 x 12
  model      mpg  cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160    110  3.9   2.62  16.5     0     1     4     4
2 Mazda RX4 ~    21     6  160    110  3.9   2.88  17.0     0     1     4     4
3 Datsun 710     22.8     4  108     93  3.85  2.32  18.6     1     1     4     1
4 Hornet 4 D~   21.4     6  258    110  3.08  3.22  19.4     1     0     3     1
5 Hornet Spo~   18.7     8  360    175  3.15  3.44  17.0     0     0     3     2
6 Valiant       18.1     6  225    105  2.76  3.46  20.2     1     0     3     1
7 Duster 360    14.3     8  360    245  3.21  3.57  15.8     0     0     3     4
8 Merc 240D     24.4     4  147.     62  3.69  3.19  20      1     0     4     2
9 Merc 230      22.8     4  141.     95  3.92  3.15  22.9    1     0     4     2
10 Merc 280     19.2     6  168.    123  3.92  3.44  18.3    1     0     4     4
# i 22 more rows

```

Lectura selectiva de ciertas columnas, con la opción `col_select =`

```
read_tsv("tibble_mtcars.tsv", col_select = "model", show_col_types = FALSE)
```

```

# A tibble: 32 x 1
  model
  <chr>
1 Mazda RX4
2 Mazda RX4 Wag
3 Datsun 710
4 Hornet 4 Drive
5 Hornet Sportabout
6 Valiant
7 Duster 360
8 Merc 240D
9 Merc 230
10 Merc 280
# i 22 more rows

```

Lectura de múltiples archivos a la vez

```
read_tsv(c("tibble_mtcars.tsv", "tibble_mtcars.tsv"), show_col_types = FALSE)
```

```
# A tibble: 64 x 12
  model      mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160   110  3.9   2.62  16.5     0    1     4     4
2 Mazda RX4 ~    21     6  160   110  3.9   2.88  17.0     0    1     4     4
3 Datsun 710    22.8     4  108    93  3.85   2.32  18.6     1    1     4     1
4 Hornet 4 D~   21.4     6  258   110  3.08   3.22  19.4     1    0     3     1
5 Hornet Spo~   18.7     8  360   175  3.15   3.44  17.0     0    0     3     2
6 Valiant      18.1     6  225   105  2.76   3.46  20.2     1    0     3     1
7 Duster 360    14.3     8  360   245  3.21   3.57  15.8     0    0     3     4
8 Merc 240D     24.4     4  147.    62  3.69   3.19   20      1    0     4     2
9 Merc 230      22.8     4  141.    95  3.92   3.15  22.9     1    0     4     2
10 Merc 280     19.2     6  168.   123  3.92   3.44  18.3     1    0     4     4
# i 54 more rows
```

5.4. Lectura de otros archivos comunes

Nota

Aunque parte del **tidyverse**, estos paquetes no se cargan automáticamente con `library(tidyverse)`, por lo que deben importarse individualmente usando `library()`.

5.4.1. Excel

```
library(readxl)
read_excel("tibble_mtcars.xlsx", col_names = TRUE)
```

```
# A tibble: 32 x 12
  model      mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160   110  3.9   2.62  16.5     0    1     4     4
2 Mazda RX4 ~    21     6  160   110  3.9   2.88  17.0     0    1     4     4
3 Datsun 710    22.8     4  108    93  3.85   2.32  18.6     1    1     4     1
4 Hornet 4 D~   21.4     6  258   110  3.08   3.22  19.4     1    0     3     1
5 Hornet Spo~   18.7     8  360   175  3.15   3.44  17.0     0    0     3     2
6 Valiant      18.1     6  225   105  2.76   3.46  20.2     1    0     3     1
7 Duster 360    14.3     8  360   245  3.21   3.57  15.8     0    0     3     4
8 Merc 240D     24.4     4  147.    62  3.69   3.19   20      1    0     4     2
9 Merc 230      22.8     4  141.    95  3.92   3.15  22.9     1    0     4     2
10 Merc 280     19.2     6  168.   123  3.92   3.44  18.3     1    0     4     4
# i 22 more rows
```

5.4.2. Google sheets

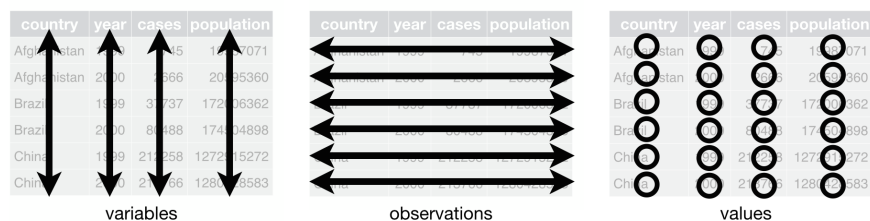
```
library(googleheets4)
read_sheet("https://docs.google.com/spreadsheets/d/1zv9rtRuGF9jbdU0QEqiSBwoJSjjT9sOBzGn4ZRS...")
```

```
# A tibble: 32 x 12
  model      mpg   cyl  disp    hp  drat    wt   qsec    vs  am  gear  carb
  <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Mazda RX4      21     6  160   110   3.9   2.62  16.5     0     1     4     4
2 Mazda RX4 ~    21     6  160   110   3.9   2.88  17.0     0     1     4     4
3 Datsun 710    22.8     4  108    93   3.85   2.32  18.6     1     1     4     1
4 Hornet 4 D~   21.4     6  258   110   3.08   3.22  19.4     1     0     3     1
5 Hornet Spo~   18.7     8  360   175   3.15   3.44  17.0     0     0     3     2
6 Valiant       18.1     6  225   105   2.76   3.46  20.2     1     0     3     1
7 Duster 360    14.3     8  360   245   3.21   3.57  15.8     0     0     3     4
8 Merc 240D     24.4     4  147.    62   3.69   3.19   20      1     0     4     2
9 Merc 230      22.8     4  141.    95   3.92   3.15  22.9     1     0     4     2
10 Merc 280     19.2     6  168.   123   3.92   3.44  18.3     1     0     4     4
# i 22 more rows
```

6. Tidy

6.1. Filosofía del tidyverse

- Cada variable es una columna; cada columna es una variable.
- Cada observación es una fila; cada fila es una observación.
- Cada valor es una celda; cada celda es un único valor.



6.2. Ejemplo de datos desordenados

```
relig_income

# A tibble: 18 x 11
  religion `<$10k` ` $10-20k` ` $20-30k` ` $30-40k` ` $40-50k` ` $50-
75k` ` $75-100k`
```

	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Agnostic	27	34	60	81	76	137	122
2	Atheist	12	27	37	52	35	70	73
3	Buddhist	27	21	30	34	33	58	62
4	Catholic	418	617	732	670	638	1116	949
5	Don't k~	15	14	15	11	10	35	21
6	Evangel~	575	869	1064	982	881	1486	949
7	Hindu	1	9	7	9	11	34	47
8	Histori~	228	244	236	238	197	223	131
9	Jehovah~	20	27	24	24	21	30	15
10	Jewish	19	19	25	25	30	95	69
11	Mainlin~	289	495	619	655	651	1107	939
12	Mormon	29	40	48	51	56	112	85
13	Muslim	6	7	9	10	9	23	16
14	Orthodox	13	17	23	32	32	47	38
15	Other C~	9	7	11	13	13	14	18
16	Other F~	20	33	40	46	49	63	46
17	Other W~	5	2	3	4	2	7	3
18	Unaffil~	217	299	374	365	341	528	407

```
# i 3 more variables: ` $100-150k` <dbl>, ` >150k` <dbl>,
# `Don't know/refused` <dbl>
```

i Nota

Este dataset no sigue la filosofía de datos ordenados. La variable correspondiente al **ingreso** está repartida a lo largo de diferentes columnas.

6.3. Transformación a lo largo: `pivot_longer()`

```
relig_income_long <- relig_income %>%
  pivot_longer(
    cols = 2:11,
    names_to = "income",
    values_to = "frequency"
  )

print(relig_income_long, n = 10)
```

```
# A tibble: 180 x 3
  religion income      frequency
  <chr>    <chr>      <dbl>
1 Agnostic <$10k          27
2 Agnostic $10-20k         34
3 Agnostic $20-30k         60
4 Agnostic $30-40k         81
```

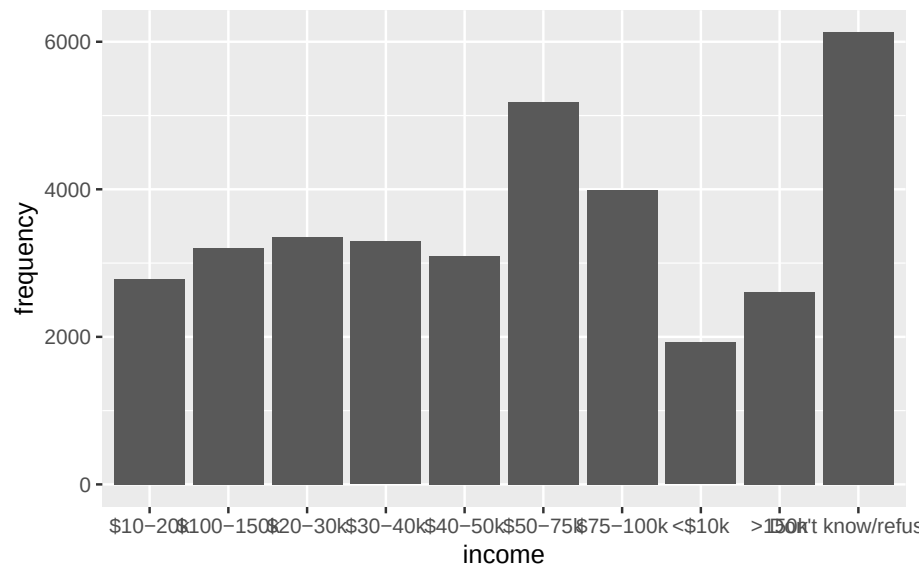
5	Agnostic	\$40-50k	76
6	Agnostic	\$50-75k	137
7	Agnostic	\$75-100k	122
8	Agnostic	\$100-150k	109
9	Agnostic	>150k	84
10	Agnostic	Don't know/refused	96

i 170 more rows

6.3.1. Ahora es más fácil de graficar

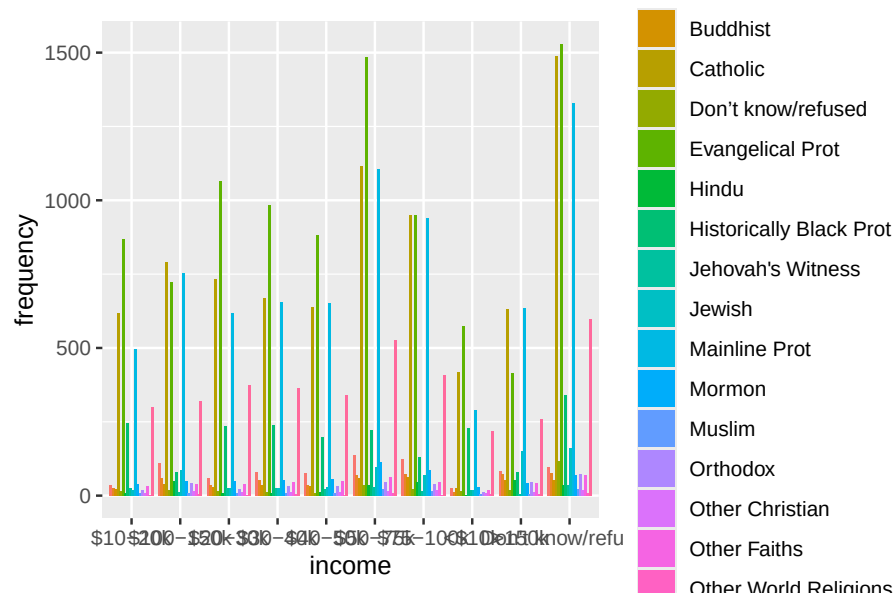
Frecuencia de salarios en la encuesta

```
ggplot(relig_income_long,
  aes(x = income, y = frequency)) +
  geom_col()
```



Frecuencia de salarios por religión en la encuesta

```
ggplot(relig_income_long,
  aes(x = income, y = frequency, fill = religion)) +
  geom_col(position = "dodge")
```



6.4. Transformación a lo ancho: `pivot_wider()`

i Nota

Opuesto a `pivot_longer()`

```
relig_income_long %>%
  pivot_wider(names_from = income,
              values_from = frequency)
```

A tibble: 18 x 11

religion `<\$10k` ` \$10-20k` ` \$20-30k` ` \$30-40k` ` \$40-50k` ` \$50-75k` ` \$75-100k`

	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Agnostic	27	34	60	81	76	137
2	Atheist	12	27	37	52	35	70
3	Buddhist	27	21	30	34	33	58
4	Catholic	418	617	732	670	638	1116
5	Don't k~	15	14	15	11	10	35
6	Evangel~	575	869	1064	982	881	1486
7	Hindu	1	9	7	9	11	34
8	Histori~	228	244	236	238	197	223
9	Jehovah~	20	27	24	24	21	30
10	Jewish	19	19	25	25	30	95
11	Mainlin~	289	495	619	655	651	1107
12	Mormon	29	40	48	51	56	112

```

13 Muslim      6      7      9      10      9      23      16
14 Orthodox    13     17     23     32     32     47     38
15 Other C~     9      7     11     13     13     14     18
16 Other F~    20     33     40     46     49     63     46
17 Other W~     5      2      3      4      2      7      3
18 Unaffil~    217    299    374    365    341    528    407
# i 3 more variables: ` $100-150k` <dbl>, ` >150k` <dbl>,
#   `Don't know/refused` <dbl>

```

6.5. Otro ejemplo de datos desordenados

Nota

Este dataset no sigue la filosofía de datos ordenados. La variable correspondiente al **radio** contiene dos valores por celda.

```
table3
```

```

# A tibble: 6 x 3
  country      year rate
  <chr>      <dbl> <chr>
1 Afghanistan 1999 745/19987071
2 Afghanistan 2000 2666/20595360
3 Brazil      1999 37737/172006362
4 Brazil      2000 80488/174504898
5 China       1999 212258/1272915272
6 China       2000 213766/1280428583

```

6.6. Separación a lo largo: separate_longer_delim()

```

table_longer <- table3 %>%
  separate_longer_delim(cols = rate, delim = "/")
table_longer

```

```

# A tibble: 12 x 3
  country      year rate
  <chr>      <dbl> <chr>
1 Afghanistan 1999 745
2 Afghanistan 1999 19987071
3 Afghanistan 2000 2666
4 Afghanistan 2000 20595360
5 Brazil      1999 37737
6 Brazil      1999 172006362
7 Brazil      2000 80488
8 Brazil      2000 174504898

```


9	China	1999	212258
10	China	1999	1272915272
11	China	2000	213766
12	China	2000	1280428583

6.7. Separación a lo ancho: `separate_wider_delim()`

```
table3_sep <- table3 %>%
  separate_wider_delim(
    cols = rate,
    delim = "/",
    names = c("rate", "population")
  )
table3_sep
```

```
# A tibble: 6 x 4
  country      year rate population
  <chr>      <dbl> <chr>   <chr>
1 Afghanistan 1999  745 19987071
2 Afghanistan 2000 2666 20595360
3 Brazil      1999 37737 172006362
4 Brazil      2000 80488 174504898
5 China       1999 212258 1272915272
6 China       2000 213766 1280428583
```

6.8. Unir variables

i Nota

Opuesto a `separate_wider_delim()`

```
table3_sep %>%
  unite(
    col = "rate",
    rate, population,
    sep = "/"
  )
```

```
# A tibble: 6 x 3
  country      year rate
  <chr>      <dbl> <chr>
1 Afghanistan 1999 745/19987071
2 Afghanistan 2000 2666/20595360
3 Brazil      1999 37737/172006362
4 Brazil      2000 80488/174504898
5 China       1999 212258/1272915272
```

7. Purr

7.1. ¿Por qué programación funcional?

La programación funcional es una forma de programar donde se usan funciones para trabajar y transformar datos. En lugar de repetir instrucciones con muchos ciclos y cambios de variables, se aplican funciones que realizan tareas específicas, haciendo que el código sea más limpio, fácil de leer y mantener.

7.2. Con programación funcional podemos sustituir ciclos

Supongamos que queremos multiplicar cada uno de los siguientes valores del vector `x` por dos

```
x <- 1:3  
x
```

```
[1] 1 2 3
```

Para calcularlo creamos el siguiente ciclo

```
for(i in x){  
  y <- i*2  
  print(y)  
}
```

```
[1] 2
```

```
[1] 4
```

```
[1] 6
```

7.2.1. También podemos crear una función con la misma utilidad

```
multiplicacion <- function(x){  
  y <- x*2  
  
  print(y)  
  
  return(y)  
}
```

Pero... ¿Cómo hacemos para que esta función se repita?

7.3. `map()`

`map()` es una forma más compacta de aplicar funciones a cada elemento de un vector o lista, devolviendo una lista. Siguiendo el ejemplo anterior, le aplicaremos

la función a cada uno de los elementos del vector `x` con ayuda de `map`.

```
x <- 1:3

multiplicacion <- function(x){
  y <- x*2

  print(y)

  return(y)
}

resultado_multiplicacion <- map(x, multiplicacion)
```

```
[1] 2
[1] 4
[1] 6
```

```
resultado_multiplicacion
```

```
[[1]]
[1] 2
```

```
[[2]]
[1] 4
```

```
[[3]]
[1] 6
```

Nota

El resultado en `resultado_multiplicacion` está contenido en forma de una lista, donde cada uno de los resultados es un vector individual. Para convertirlo en un único vector podemos usar la función `unlist` sobre `resultado_multiplicacion`

7.4. `walk()`

A diferencia de `map` donde la función está enfocada en almacenar los resultados de aplicar la función a un vector o una lista, `walk` está enfocada en los efectos colaterales de aplicar dicha función, no en guardar los resultados. Los efectos colaterales pueden ser imprimir resultados de una operación en la consola, guardar una tabla o leer un archivo.

```
x <- 1:3

multiplicacion <- function(x){
```

```

    y <- x*2

    print(y)

    return(y)
}

walk(x, multiplicacion)

```

```

[1] 2
[1] 4
[1] 6

```

Si guardamos los resultados de una función aplicada con `walk` se guardará el valor del vector al que le aplicamos la función, en este caso `x`

```

resultado_multiplicacion_walk <- walk(x, multiplicacion)

```

```

[1] 2
[1] 4
[1] 6

```

```

resultado_multiplicacion_walk

```

```

[1] 1 2 3

```

7.5. `map2` y `walk2`

Con las funciones `map2` y `walk2` se aplica la función a un par de elementos de dos vectores o listas, de igual forma devolviendo una lista.

Supongamos que queremos calcular el cuadrado de `x`

```

x <- 1:3

cuadrado <- function(x,y){
  z <- x*y

  print(z)

  return(z)
}

resultado_cuadrados <- map2(.x = x, .y = x, cuadrado)

```

```

[1] 1
[1] 4
[1] 9

```

```
resultado_cuadrados
```

```
[[1]]  
[1] 1
```

```
[[2]]  
[1] 4
```

```
[[3]]  
[1] 9
```

```
walk2(.x = x, .y = x, .f = cuadrado)
```

```
[1] 1  
[1] 4  
[1] 9
```

7.6. pmap y pwalk

Con las funciones `pmap` y `pwalk` se aplica la función a un grupo de elementos, ya sean listas o listas de vectores, de igual forma devolviendo una lista.

Supongamos que queremos calcular el cubo de `x`

```
x <- 1:3
```

```
cubo <- function(x,y,z){  
  w <- x*y*z  
  
  print(w)  
  
  return(w)  
}
```

```
resultado_cubos <- pmap(.l = list(x, x, x), .f = cubo)
```

```
[1] 1  
[1] 8  
[1] 27
```

```
resultado_cubos
```

```
[[1]]  
[1] 1
```

```
[[2]]  
[1] 8
```

```
[[3]]  
[1] 27  
  
pwalk(.l = list(x, x, x), .f = cubo)
```

```
[1] 1  
[1] 8  
[1] 27
```

Nota

`pmap` y `pwalk` se usan para aplicar una función a 3 o más elementos, ya sean listas o listas de vectores

7.7. Seguimiento de progreso

El paquete `purrr` nos permite rastrear el progreso de aplicar alguna de sus funciones, con la opción `.progress = TRUE`

```
walk(1:100, \(i) Sys.sleep(0.5), .progress = TRUE)
```

Nota

La función anterior no tiene efecto alguno, su función únicamente es recorrer los números del uno al cien esperando medio segundo entre cada número, únicamente es útil para observar cómo es el seguimiento del progreso.

8. Bibliografía:

«Tidyverse». Accedido 13 de mayo de 2026. <https://www.tidyverse.org/>.